FlashFuser: Expanding the Scale of Kernel Fusion for Compute-Intensive Operators via Inter-Core Connection

Ziyu Huang[1,2,*], Yangjie Zhou[3,*], Zihan Liu[1,2,¶], Xinhao Luo[1,2], Yijia Diao[1,2], Minyi Guo[1], Jidong Zhai[4], Yu Feng[1], Chen Zhang[1], Anbang Wu[1], Jingwen Leng[1,2,¶]
[1]Shanghai Jiao Tong University [2]Shanghai Qi Zhi Institute
[3]National University of Singapore [4]Tsinghua University
[*]*Equal contribution* [¶]*Corresponding authors*
{huang_ziyu, altair.liu, lxh666, diao_yijia, guo-my, y-feng, chenzhang.sjtu, anbang, leng-jw}@sjtu.edu.cn,
yj_zhou@nus.edu.sg, zhaijidong@tsinghua.edu.cn

2026 IEEE International Symposium on High Performance Computer Architecture (HPCA) | 979-8-3315-9302-5/26/$31.00 ©2026 IEEE | DOI: 10.1109/HPCA68181.2026.11408495

***Abstract*—The scaling of computation throughput continues to outpace improvements in memory bandwidth, making many deep learning workloads memory-bound. Kernel fusion is a key technique to alleviate this problem, but the fusion strategies of existing compilers and frameworks are limited to using local scratchpad memory. When the intermediate results exceed the limited capacity (such as FFN), the fusion fails. Although modern GPUs (like the NVIDIA H100) now incorporate an inter-core connection mechanism known as Distributed Shared Memory (DSM)—providing a larger, high-bandwidth, and low-latency on-chip memory pool—this hardware potential has yet to be exploited by software frameworks.**

To bridge this gap, we present FlashFuser, the first compiler framework to utilize inter-core connection for kernel fusion on modern GPUs. FlashFuser extends established fusion techniques to the DSM domain through three core contributions. First, we propose a powerful DSM-based communication abstraction that formalizes complex cluster-based data exchange patterns, such as reduce, shuffle and multiply. Second, we introduce a dataflow analyzer that generalizes loop scheduling, resource mapping, and tile selection to the distributed memory hierarchy; it determines the optimal execution order and tile sizes by quantifying data movement across memory levels. Finally, FlashFuser integrates these components into a unified search engine that employs analytical cost modeling and DSM-aware pruning strategies to efficiently discover the optimal execution plan. Our evaluation on an NVIDIA H100 GPU shows that FlashFuser reduces memory access by 58% and delivers kernel speedups of 3.3x against highly-tuned libraries and 4.1x against state-of-the-art compilers, resulting in a 1.24× end-to-end speedup.

I. Introduction

With the rapid evolution of deep learning techniques [2], [8]–[11], [14], [23], [24], [52], [64]–[66] and the expanding scale of deep learning models, the growing inference demands from multi-modal and large language models (LLM) mean that memory bandwidth is increasingly struggling to keep up with the growth of computational power.

In new generation GPU, H100, the peak FP16 compute capability has increased to approximately 1000 TFLOPS from the 300 TFLOPS of the previous-generation A100 (3.3× increase), while the global HBM bandwidth has only grown from 2 TB/s to 3 TB/s (1.5× increase) [1], [4], [5], [26].

计算能力提升了但是hbm访存没变

TABLE I: Percentage of Execution Time Spent in FFN Layers across Different Models.

Model	FFN Time (%)
GPT-6.7B	61.28
LLaMA-1B	57.44
OPT-1.3B	53.08
BERT	47.03
GPT-2	41.64

This disparity, known as the memory wall, in growth rates makes memory bandwidth a primary bottleneck. In workloads dominated by General Matrix Multiplication (GEMM), such as Transformer Feed Forward Network (FFN) layers and convolutional blocks, insufficient HBM bandwidth often becomes a significant bottleneck. As shown in Table I, under a typical inference configuration with a sequence length of 512, the FFN in various models consumes 40%–60% of the total execution time [47] and exhibits memory-bound characteristics.

To mitigate the aforementioned bandwidth bottleneck, modern GPUs such as the H100 have introduced inter-core connected architecture, which provides a high-speed data exchange path known as Distributed Shared Memory (DSM) within a cluster composed of multiple Streaming Multiprocessors (SMs) [33]. The traditional approach relies on a costly round-trip path through global memory, whereas our approach leverages DSM to open up a direct on-chip path. This shift in the data path has two benefits. First, by avoiding the redundant "write-then-read" operation, the total volume of data transferred to and from global memory is significantly reduced. Second, this direct on-chip path provides both higher bandwidth and lower latency than global memory access [26].

Kernel fusion is an effective method for addressing the aforementioned memory-bound problem. However, current kernel fusion techniques fail to fuse large-scale operator chains. Existing software frameworks—including libraries like `cuBLAS` [32] and `CUTLASS` [41], inference frameworks

Authorized licensed use limited to: INSTITUTE OF COMPUTING TECHNOLOGY CAS. Downloaded on October 13,2026 at 09:14:11 UTC from IEEE Xplore. Restrictions apply.

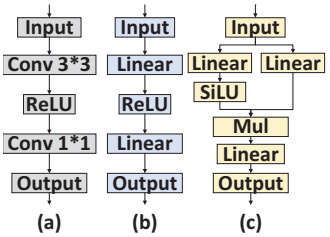


Fig. 1: Three common GEMM chains: (a) conv, (b) standard FFN, (c) gated FFN.

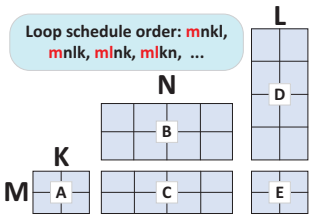


Fig. 2: A fused GEMM operator chain, showing its loop dimensions (M, N, K, L) and possible execution orders (mnkl, etc.).

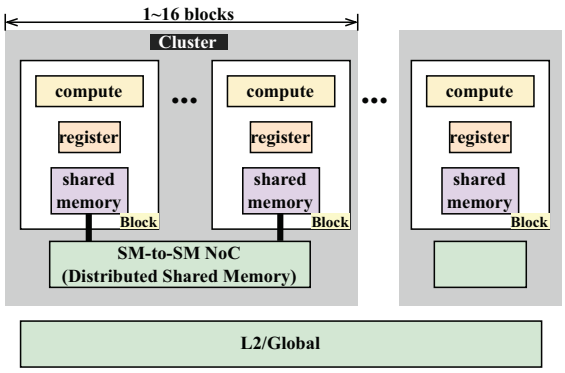


Fig. 3: The memory hierarchy of the H100 GPU, including registers, SMEM, DSM, L2 cache, and global memory.

like `SGLang` [57], or compilers like Chimera [60], BOLT [51]—typically handle smaller operator chains by placing intermediate results in the shared memory (SMEM) or registers(reg) of a single SM. When the intermediate data becomes larger (like FFN), these methods will abandon fusion, resorting to an inefficient round-trip to global memory. The inter-core connection mechanism can effectively alleviate this constraint. By interconnecting the SMEM of multiple SMs, it creates what can be viewed as an expanded on-chip memory pool. However, the complex communication patterns required to leverage this capability remains an unexplored domain.

To bridge the gap between new hardware features and existing software frameworks, we propose FlashFuser, the first deep learning (DL) compiler to leverage DSM for kernel fusion on compute-intensive operator chains. By creatively introducing DSM, FlashFuser expands the scope of fusible operators. It progressively places intermediate results to on-chip memory, including reg, SMEM, and DSM, thereby introducing a vast search space. Through corresponding pruning rules and cost model, it finds an execution order that minimizes data movement, thus achieving a performance improvement.

We use compute intensive operator chains from various LLM and CNNs for performance evaluation, running on an NVIDIA H100 GPU. FlashFuser achieves a speedup of up to 4.1x over the state-of-the-art (SOTA) baseline. In summary, the contributions of this paper are as follows:

- We identify the operator fusion bottleneck caused by SMEM capacity limitations, and point out the widespread deficiency in the current software ecosystem in utilizing inter-core connection property (§III).
- We propose a new abstraction to describe inter-core communication patterns, enabling it to support the various requirements of kernel fusion (§IV-A).
- We propose a dataflow analyzer that quantifies data-movement cost across the memory hierarchy and schedules data to spill progressively from fast to slow caches (§IV-B).
- We present a fusion search engine that employs pruning techniques and an analytical cost model to efficiently navigate the greatly expanded search space introduced by DSM (§IV-C).
- Compared to highly-tuned libraries and state-of-the-art compilers, our method delivers kernel speedups of 3.3× and 4.1×, respectively, along with a 58% reduction in memory access. These kernel-level improvements result in a 1.24× end-to-end speedup, validating the effectiveness of our approach (§VI).

II. Background

Mainstream LLM and Convolutional Neural Networks (CNNs) consist of numerous tensor operators, which are often organized into chains. As shown in Figure 1, these include convolution blocks that can be converted to GEMM chains via `im2col` (a), standard FFN (b), and Gated FFNs with branched structures (e.g., SwiGLU) (c). Due to their data-intensive nature, these GEMM-based operator chains are often limited by memory bandwidth, which makes kernel fusion a key optimization method. Figure 2 shows an example of a GEMM chain, where the dimensions of each matrix are marked as M, N, K, and L. In parallel computing, we need to split the tensors into small blocks and then iterate through these blocks according to different iteration orders. This traversal order is called a loop schedule, and as shown in the Figure 2, can be `mnkl`, `mnlk`, etc.

DSM is one tier in the multi-level cache hierarchy. Figure 3 illustrates the entire cache hierarchy of the H100 GPU. The innermost cache is the L0 cache [38], also known as the register file (reg). This is the fastest cache, but it is only visible to each thread and has a small capacity. The next tier is the L1 cache, or SMEM [34], where all threads within a single compute core can access the values in SMEM. Starting from the H100, the SMEMs of different SMs can be connected via DSM, which is also considered an L1.5 cache [6], [15], [16]. Only cores within a single cluster can exchange data, different clusters cannot directly interact and must exchange data through the next tier, the L2 cache and the global memory.

III. Motivation

The gap between new hardware features and the software ecosystem is characterized by two core limitations in existing

Authorized licensed use limited to: INSTITUTE OF COMPUTING TECHNOLOGY CAS. Downloaded on October 13,2026 at 09:14:11 UTC from IEEE Xplore. Restrictions apply.

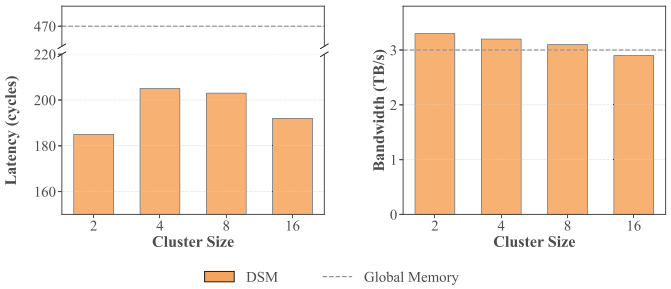


Fig. 4: Bandwidth and latency of DSM under different cluster sizes. The corresponding performance of global memory is marked in the figure for comparison.

TABLE II: Comparison of FlashFuser with representative previous works. Cache Hierarchy 0,1,1.5 means register(reg), shared memory(SMEM) and dsm

Framework	Cache Hier.	Strategy	GPU Supp.	Fusion
BOLT [51]	0/1	Tuning	yes	yes
Chimera [60]	1	Analytical	yes	yes
Welder [40]	0/1	Analytical	yes	yes
MCFuser [55]	1	Analytical	yes	yes
T10 [22]	1/1.5	Analytical	no	no
WaferLLM [12]	1/1.5	Handcrafted	no	no
Ours	**0/1/1.5**	**Analytical**	**yes**	**yes**

works:

(a) Fusion is constrained by on-chip memory capacity: Current kernel fusion frameworks are constrained by SMEM capacity, which prevents the fusion of large-scale operator chains. Current frameworks only consider reg and SMEM for data reuse and make the overly simplistic assumption that intermediate results can always be accommodated on-chip; however, this assumption does not hold true in many scenarios. As illustrated in Figure 5, while Chimera [60] can still store intermediate results on-chip when memory usage is relatively small, it encounters fusion failures when executing larger-scale GEMM chains, such as those in OPT1_3B and GPT6_7B. As shown by the purple dotted line in the figure, the upper limit of SMEM for a single SM on H100 is 227KB. When the intermediate result exceeds this size, the fusion will fail.

(b) DSM can expand on-chip memory, however, its performance is non-trivial: By connecting multiple SMs, DSM provides an effectively larger SMEM space, which can solve the problem of SMEM size limitation. However, its complex characteristics make it difficult to utilize directly. **Firstly,** the bandwidth and latency of DSM vary with cluster size. As shown in Figure 4, as the cluster size increases, its latency tends to increase while its bandwidth gradually decreases. For all cluster sizes, the DSM latency is lower than that of global memory, and for all but the largest cluster size, its bandwidth is faster [18], [25], [26], making the selection of an appropriate cluster size a non-trivial problem. **Secondly,** the introduction of clusters adds another layer to the memory hierarchy, making dataflow more complex. Since prior works did not incorporate DSM, they are unable to analyze how data should be placed on DSM or the resulting data movement volume across various cache levels. This analysis involves crucial details such as how tiles are partitioned, their execution order, their sizes, and resource mapping. **Finally,** the introduction of DSM makes many previously infeasible fusion scenarios possible. This is because considering DSM is equivalent to expanding the on-chip memory space, making more strategies feasible that would have been directly pruned in prior works. As detailed in Section IV-C, for GPT6_7B, the number of possibilities with traditional methods [55] after pruning is approximately 10^4, whereas with DSM, this expands to 10^6. Therefore, an analysis framework specifically targeted for DSM is essential.

Prior works, as summarized in Table II, have only partially addressed the aforementioned issues. Chimera [60] and MCFuser [55] considered how to fuse GEMM chains, but because they only use SMEM to store intermediate results, they are severely limited by the SMEM capacity of a single SM and thus cannot be used for scenarios with larger intermediates, such as FFNs. BOLT [51] considered how to use registers or SMEM to perform GEMM chain fusion; however, it did not consider different computation orders and used manual tuning to find parameters, meaning its search results are not necessarily optimal. Welder [40] used an analytical method to explore data reuse for reg and SMEM, but it also did not consider DSM. Previous papers on DSM, such as T10 [22] and WaferLLM [12], are works on Graphcore and Cerebras, respectively; both utilized inter-core connect features but did not consider kernel fusion, nor were they explored on GPUs.

IV. Design

We now introduce FlashFuser, a compiler designed to optimize kernel fusion for operator chains on processors with inter-core connection. An overview of FlashFuser is presented in Figure 6:

(1) FlashFuser defines a DSM-communication primitive that compactly encodes SM partitioning and inter-SM dataflow, yielding a unified representation of DSM-based fusion plans under the given model and hardware (see §IV-A).

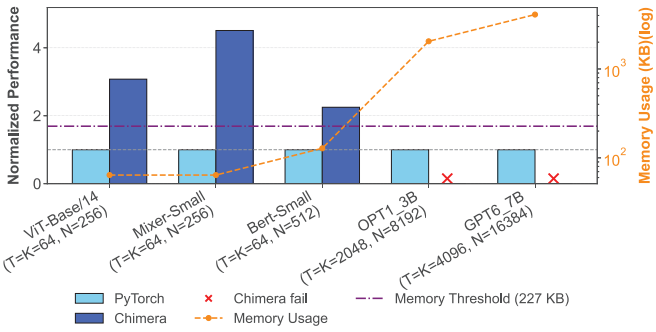


Fig. 5: Relative performance of Chimera to torch. The workload consists of two consecutive GEMM operations. M is set as 128 here.

Authorized licensed use limited to: INSTITUTE OF COMPUTING TECHNOLOGY CAS. Downloaded on October 13,2026 at 09:14:11 UTC from IEEE Xplore. Restrictions apply.

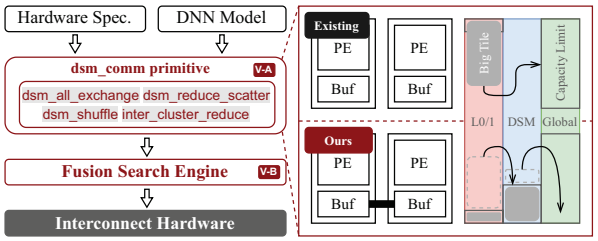


Fig. 6: System overview of FlashFuser

(2) Based on this representation, our dataflow analyzer evaluates the feasibility and cost—in terms of data movement volume—of any given plan. It models the entire on-chip memory hierarchy, determining how intermediate data is progressively spilled from high-speed caches to slower tiers (like DSM) when capacity is exceeded (see §IV-B).

(3) The incorporation of DSM unlocks many new fusion possibilities, thereby creating an enormous search space. To navigate this, FlashFuser employs a fusion search engine. Guided by a cost model and a set of pruning rules, the engine efficiently searches for the optimal execution plan (see §IV-C).

Methodologically, FlashFuser adapts established techniques from prior kernel fusion works, including loop scheduling, tile selection, and resource mapping (see §IV-B), as well as cost modeling and pruning (see §IV-C). **However, our fundamental novelty lies in integrating DSM into these methods**—specifically, by introducing DSM-level tiling, accounting for DSM bandwidth variations across cluster sizes, and respecting maximum cluster size limits, etc.

A. *dsm_comm primitive*

Conventional GPU programming models have primarily focused on a single tiling hierarchy at the thread block level. The introduction of DSM necessitates a higher-level tiling at the thread block cluster level, which in turn requires explicit handling of intra-cluster and inter-cluster communication.

To elaborate on this design, we use a fused kernel containing two GEMM operations as an example. The execution of this fused kernel is divided into three distinct phases: $GEMM_0$, $GEMM_1$, and Store phase (as illustrated in Figure 7). In this context, a bold rectangle denotes a Cluster, a non-bold rectangle denotes a Block, and a rounded rectangle represents a Shuffle Group, where Blocks within it perform shuffle operations to exchange data. We define two base parameters: cls_i, representing the number of parallel Blocks within a Cluster along dimension i, and blk_i, representing the data granularity computed by a Block along dimension i.

Crucially, the dataflow between blocks in the GEMM chain is uniquely determined by the declared cluster size. In the two-GEMM scenario, these dimensions correspond to cls_m, cls_n, cls_k, and cls_l. As shown in Figure 7(a), the cluster size is (2, 4, 2, 4). In the $GEMM_0$ phase, $cls_k = 2$ signifies that the K-dimension is spatially partitioned across two parallel Blocks. Consequently, these Blocks must perform an intra-cluster accumulation along the K-dimension. We introduce the `dsm_all_exchange` primitive for this purpose, ensuring each Block holds the complete, fully-accumulated intermediate result before proceeding.

In the $GEMM_1$ and Store phases, data must be shared among the Blocks to compute the final matrix E. We employ two complementary strategies—shuffle and reduce—to compute matrix E. The first strategy is a shuffle, where data is exchanged during the $GEMM_1$ computation. As shown in Figure 7(a), calculating one Block of matrix E requires access to an entire row of data from matrix C. Therefore, Blocks within the same Shuffle Group exchange their respective slices of matrix C using the `dsm_shuffle` primitive.

The second strategy is reduce, which postpones data exchange to the final Store phase. Here, each Block first independently computes a partial sum of the output matrix E. This is followed by a two-level hierarchical reduction. The intra-cluster reduction is performed first, where multiple contributing Shuffle Groups perform an accumulation via the `dsm_reduce_scatter` operation. The Scatter pattern is employed because each Block is only responsible for writing back a portion of the final result, thus avoiding data redundancy. This is followed by the `inter_cluster_reduce`, which aggregates partial sums from all participating clusters. This step is implemented by leveraging the NVIDIA Hopper architecture's Tensor Memory Accelerator (TMA). Through its `cp.reduce.async.bulk` instruction, the TMA can asynchronously perform atomic reductions across clusters.

To precisely describe these communication patterns, we derive two key variables based on the established cluster parameters: cls_{shuffle}, the number of Blocks within a single Shuffle Group, and cls_{reduce}, the number of Shuffle Groups participating in a Reduce operation. Their derivations are $cls_{\text{shuffle}} = cls_l/cls_k$ and $cls_{\text{reduce}} = cls_n/cls_{\text{shuffle}} = (cls_n \times cls_k)/cls_l$. For instance, Figure 7(b) illustrates an alternative configuration where the `cls size` is (2, 4, 2, 8). Here, $cls_{\text{reduce}} = 1$, meaning no inter-group reduction is needed during the Store phase. The trade-off is that the larger cls_{shuffle} increases the communication volume for the shuffle operations, while decreasing the number of required `dsm_reduce_scatter` operations. Moreover, this flexibility to configure the shuffle and reduce dimensions is crucial for efficiently mapping problem sizes that are small or not perfectly divisible onto the hardware.

Based on the aforementioned DSM communication primitives, we can abstract complex fused kernels into an intuitive tile graph to describe the dataflow. As shown in Figure 8, the graph clearly demonstrates the flexibility of our framework: it can support not only standard FFNs composed of consecutive GEMMs (Figure 8(a)) but also the more structurally complex Gated FFN variant (Figure 8(b)).

We use the standard FFN in Figure 8(a) to illustrate the detailed dataflow. A symbol such as $B_{0,0}$ denotes the tile of matrix B at coordinate (0,0). The process begins with tiles like $A_{0,0}$ and $B_{0,0}$ being multiplied to produce a partial sum of the intermediate matrix C, denoted as $C_{0,0}(0)$. After all

Authorized licensed use limited to: INSTITUTE OF COMPUTING TECHNOLOGY CAS. Downloaded on October 13,2026 at 09:14:11 UTC from IEEE Xplore. Restrictions apply.

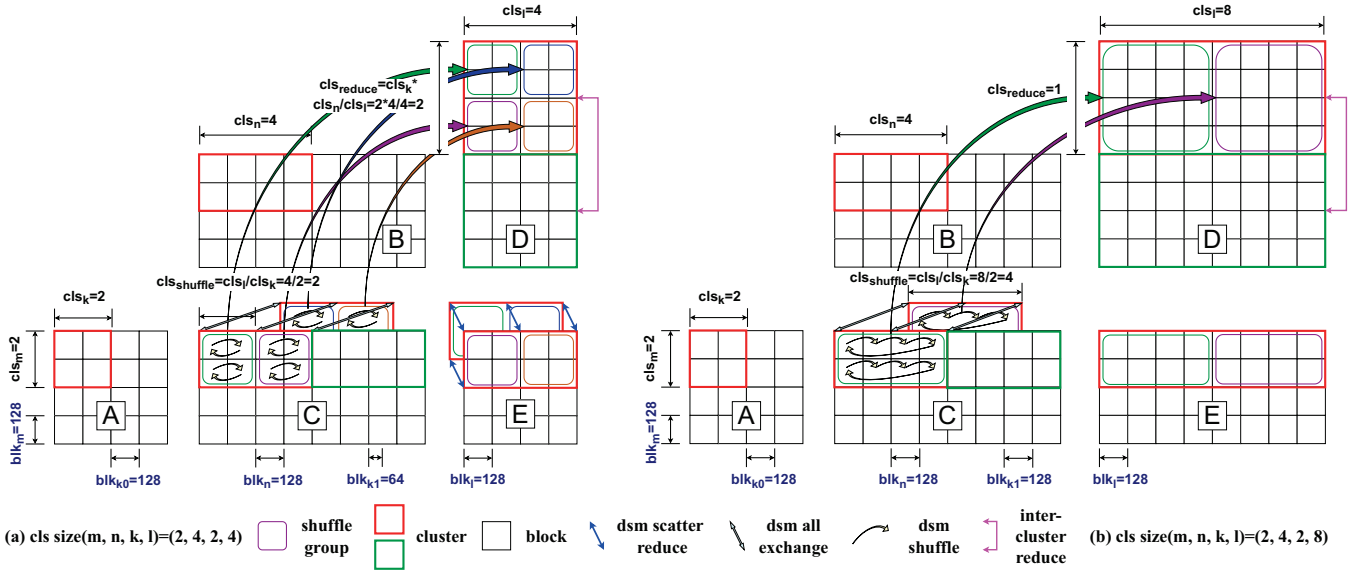


Fig. 7: Conceptual illustration of the cluster and tile geometries.

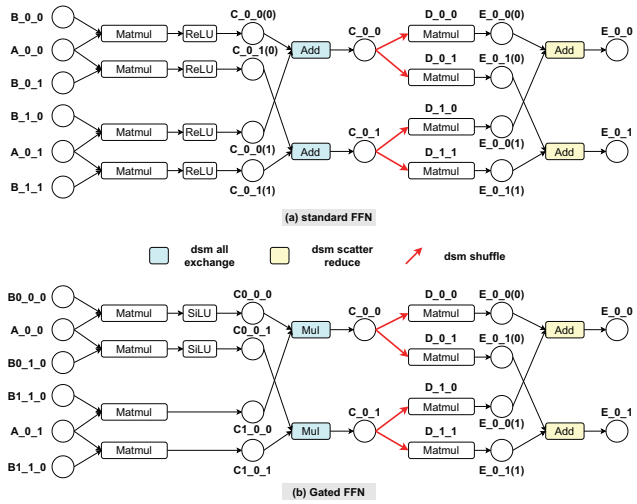


Fig. 8: Tile graph of kernel fused with the `dsm_comm` primitive (only show one cluster). (a) standard FFN. (b) gated FFN

parallel partial sums (e.g., $C_{0,0}(0)$ and $C_{0,0}(1)$) are computed, the `dsm_all_exchange` primitive performs an All-Reduce operation within the cluster to produce the complete intermediate tile $C_{0,0}$. Subsequently, $C_{0,0}$ serves as input to GEMM1 and is distributed by the `dsm_shuffle` primitive (red arrows) to different compute units to be multiplied with different tiles of matrix D, yielding new partial sums for matrix E (e.g., $E_{0,0}(0)$ and $E_{0,1}(0)$). Finally, during the Store phase, these partial sums of E are accumulated by the `dsm_scatter_reduce` primitive to obtain the complete output tiles $E_{0,0}$ and $E_{0,1}$.

For the Gated FFN in Figure 8(b), the core difference is that its Up-FFN portion executes two parallel GEMM branches, where the result of one branch, after a SiLU activation, is element-wise multiplied with the other. This impacts the function of the first DSM primitive: `dsm_all_exchange` now performs a `Mul` operation instead of an `Add`, which is why we chose the generic name "exchange" to reflect its operational flexibility. To implement the two parallel GEMM branches, our framework supports two approaches. The first is to leverage spatial partitioning by setting `cls_k = 2`, which assigns the two GEMM branches to different groups of Blocks. The final element-wise multiplication is then performed by the `dsm_all_exchange` primitive, which executes a `Mul` operation to combine the results. The second approach is to execute the two GEMMs sequentially within a single Block, which effectively transforms the computation into the pattern of a standard FFN, but with a doubled K-dimension.

The two approaches allow for optimizing different goals: the first, spatial partitioning across the cluster, is designed to maximize parallelism, while the second, sequential execution within each Block, aims to minimize DSM communication overhead.

B. Dataflow Analyzer

After introducing the `dsm_comm` primitive, we incorporate it into our **Dataflow Analyzer**. This analyzer is designed to tackle the complex dataflow challenges introduced by inter-core connection. While conventional methods only need to consider the register and SMEM hierarchy, our approach must also orchestrate the newly introduced DSM tier. To address this, FlashFuser employs a tile-based analysis method. For a given set of parameters, the analyzer determines how to efficiently place intermediate data for reuse across the memory levels. Crucially, it also analyzes the data movement in detail, allowing it to calculate critical performance costs, such as the data transfer volume for each tier of the memory hierarchy.

Authorized licensed use limited to: INSTITUTE OF COMPUTING TECHNOLOGY CAS. Downloaded on October 13,2026 at 09:14:11 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Dataflow Analyzer

Input : Graph g, Device d,
Loop schedule $s = \{s_1, s_2, \ldots, s_x\}$,
Tile sizes $t = \{t_1, t_2, \ldots, t_x\}$,
Initial resource mapping r
Output: Data movement volume D_V, Final plan p_{final}

1 **Function** `DataflowAnalyzer` (g, d, s, t, r) **:**
2 $mapping_plan \leftarrow$ new ResourceMapping();
3 $D_V \leftarrow 0$;
4 $hierarchy \leftarrow d$.getMemoryHierarchy();
5 $S \leftarrow g$.getDimensionSizes();
6 **foreach** *tensor in* $g.tensors()$ **do**
7 $DF \leftarrow$ `GetFootprint`$(t.block)$
8 **if** $tensor \in g.IOTensors()$ **then**
9 $DM \leftarrow DF$;
10 **foreach** s_i *in reversed(s)* **do**
11 **if** s_i *accesses tensor* **then**
12 $DM \leftarrow DM \times \lceil S_i / t_i.block \rceil$
13 $D_V[global] \leftarrow D_V[global] + DM$;
14 **else**
15 $remaining \leftarrow DF$;
16 $mapping \leftarrow$ new TensorMapping();
`// Greedily place tensor across memory hierarchy`
17 **foreach** *level in hierarchy* **do**
18 **if** $remaining \leq 0$ **then**
20 **break**;
21 $alloc \leftarrow \min(remaining, level.capacity)$;
22 $mapping[level] \leftarrow alloc$;
23 $remaining \leftarrow remaining - alloc$;
24 **foreach** s_i *in reversed(s)* **do**
25 **if** s_i *accesses tensor* **then**
26 $D_V[\text{level}] \leftarrow update_dv(t_i.cluster, DF)$;
27 $mapping_plan[tensor] \leftarrow mapping$;
28 $p_{final} \leftarrow (s, t, mapping_plan)$
29 **return** (D_V, p_{final});

1) Loop Scheduling: The `LoopSchedule` defines the loop execution order for a operator chain. First, we unify the codependent loop dimensions from all operators into a single independent set, formally denoted as $\mathscr{X} = \{x_0, x_1, \ldots, x_{J-1}\}$. This set is then scheduled by defining a permutation s to set the nesting order and partitioning the dimensions into *spatial* ($\mathscr{S}$) or *temporal* ($\mathscr{T}$). *Spatial* refers to using multiple parallel processing units to compute a dimension simultaneously, while *temporal* refers to using a single processing unit to sequentially compute an entire dimension over time.

Different loop schedules affect the size of the tensor that needs to be cached. As illustrated in Figure 9, the MLNK order in (a) requires the local block to store the complete tensor C. Depending on the hardware speculation and problem size, this may require spilling from reg to SMEM, or further to DSM. In contrast, the MNLK order in (b) produces a partial E after each iteration of the LK loops. Although accumulating in registers

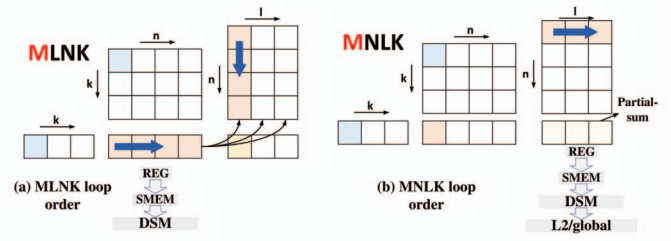


Fig. 9: An example of the hierarchical spilling plan, illustrating different spilling strategies. The red 'M' denotes the spatial dimension, while the black letters represent temporal dimensions.

is most efficient, the limited register space may necessitate spilling to SMEM, DSM, or even L2/global.

2) Tile Selection: The tile size is defined across three hierarchical levels: a cluster-level vector (`tile.cluster`) that dictates how work is distributed across clusters, a block-level vector (`tile.block`) that governs the tile size computed by each block.

This tiling directly impacts memory usage and dataflow patterns. The block-level factors (`tile.block`) determine the data tile size each thread block must hold, influencing the decision of whether to use registers or shared memory. The cluster-level factors (`tile.cluster`) influence data distribution across SMs, thereby determining whether intermediate data must spill to global memory and dictating the inter-block data exchange patterns.

3) Resource Mapping: Our framework binds tensors to different memory tiers through a heuristic-driven approach. This process, detailed in Algorithm 1, analyzes the memory usage of reusable tensor —as determined by the `LoopSchedule` and `TilingSize`—and then generate its data reuse plan across the cache hierarchy.

We use the `DataflowAnalyzer` to generate a concrete spilling plan for reused tensor. This function takes a computation graph (g), device information (d), a loop schedule (s), a tile size (t), and an initial resource mapping (r) as its inputs.

First, we obtain the size of each dimension (M, N, K, L) from the graph. If a dimension is spatial, a full traversal is not required, so its effective size is set to the corresponding tile size (line 5). We use the 'getFootprint' function to obtain the data access volume within a single tile (line 7). For input and output tensors, we calculate their total data movement volume from global memory. This is achieved by iterating through all dimensions; for each dimension relevant to the tensor, the data movement is multiplied by a factor that accounts for the increased accesses caused by tiling (lines 8-13). For a reused tensor, it is not necessarily placed in a single memory level; it can be distributed across multiple levels. Its data footprint (DF) determines the required memory size. A greedy algorithm is then employed to place the tensor on the highest-level memory possible. If a level's capacity is exceeded, the remaining portion is spilled to the subsequent level in the hierarchy (lines 17-23). Throughout this process, the data movement volume

Authorized licensed use limited to: INSTITUTE OF COMPUTING TECHNOLOGY CAS. Downloaded on October 13,2026 at 09:14:11 UTC from IEEE Xplore. Restrictions apply.

TABLE III: Pruning results based on rules

Pruning Step	# of Cand.	Reduc. Rate
Original Space	$\approx 2.75 \times 10^{13}$	-
+ Rule 1	$\approx 1.14 \times 10^{8}$	$> 99.99\%$
+ Rule 2	$\approx 2.47 \times 10^{7}$	78.3%
+ Rule 3	$\approx 1.44 \times 10^{7}$	41.5%
+ Rule 4	$\approx 9.62 \times 10^{6}$	33.3%
+ Rule 5	$\approx 1.15 \times 10^{6}$	88.0%
Total Reduction		$> 99.99\%$

for each cache level is calculated. Since DSM has a lower bandwidth than SMEM, our analysis primarily focuses on the DSM traffic. As described in `dsm_comm`, we calculate the DSM traffic for either Standard FFN or Gated FFN based on the cluster size and data footprint, thereby deriving the data movement volume(lines 23-26). Finally, the algorithm outputs the total data movement volume and the final plan, which consists of the determined resource mapping, together with loop schedule and tile size (lines 27-29).

C. Fusion Search Engine

Our search engine is designed to efficiently explore the vast search space composed of loop schedules, tiling sizes, and resource mapping to find the optimal fusion plan. Its core principle is to leverage an analytical cost model and pruning strategies to rapidly filter out a large number of inefficient or incorrect candidates.

1) Cost Model: Our performance model is inspired by the analytical model in Chimera [60]. We model the data movement cost across the L levels of the memory hierarchy. The cost C_l of transferring data to level l is determined by the required data volume V_l for a given tiling strategy $\mathscr{T}_l$, and the memory bandwidth B_l of that level.

$$C_l(\mathscr{T}_l) = \frac{V_l(\mathscr{T}_l)}{B_l} \tag{1}$$

To optimize the overall performance, we aim to minimize the bottleneck, which is the slowest data movement stage among all memory levels. This is formulated as a minimax optimization problem:

$$\min_{\mathscr{T}_1,\ldots,\mathscr{T}_L} \left\{ \max_{l=1,\ldots,L} (C_l(\mathscr{T}_l)) \right\} \tag{2}$$

The optimization is subject to memory capacity constraints of each level, where the memory usage U_l dictated by the tiling strategy $\mathscr{T}_l$ cannot exceed the available capacity Cap_l.

$$\text{s.t.} \quad U_l(\mathscr{T}_l) \le \text{Cap}_l, \quad \forall l \in \{1,\ldots,L\} \tag{3}$$

2) Pruning Strategies: While prior work has established pruning principles for kernel fusion, these do not address the vast search space introduced by clusters and are thus insufficient for our needs. Building upon these foundations, we propose the following pruning strategies:

- **Initial Search Space:** We construct our initial search space starting from the loop schedule and tile size. Drawing from methodologies in existing work, the minimum block size is set to that of a single MMA operation, i.e., $16 \times 16 \times 16$. The cluster dimension can be chosen from one of five values $\{1,2,4,8,16\}$. Since there are 4 independent dimensions, this results in 5^4 possibilities for the cluster configuration. For a model like GPT-6.7B, we consider a problem size with $M = 256, N = 16384$, and $K = T = 4096$. The number of valid tile choices is thus $(256/16) \times (16384/16) \times (4096/16) \times (4096/16)$. As shown in Table IV, there are a total of $(24+12+4+1) = 41$ possible combinations for spatial and temporal partitioning. Therefore, the initial search space contains $(24+12+4+1) \times 5^4 \times (256/16) \times (16384/16) \times (4096/16) \times (4096/16) \approx 2.75 \times 10^{13}$ possibilities.
- **Rule 1, Divisible Tile Sizes:** This is a pruning strategy from prior work [55], which mandates that the selected tile sizes should be hardware-aware and the problem size dimensions are evenly divisible by them.
- **Rule 2, Cluster Size Constraint:** The product of cluster dimensions for each GEMM across M, N, and K must be less than the hardware limit (for H100, it is 16), and the cluster dimensions of consecutive GEMMs must be identical to ensure feasibility.
- **Rule 3, Activation constraint:** To ensure the correctness of the activation between consecutive GEMMs, the accumulation dimension of preceding GEMM must be placed in the innermost loop. Otherwise, partial sums would be computed, which cannot be used by the activation and would lead to incorrect results in the subsequent GEMM.
- **Rule 4, Dependency constraint:** If L dimension is set as spatial, given the dependency of GEMM, all spatial tile in L dimension will need intermediate tensor of C, but different tiles can not communicate with each other directly, therefore the fusion will fail.
- **Rule 5, Memory Capacity Limit:** A tensor cannot exceed the capacity of the lowest-level cache to which it can spill.

Among the rules above, only Rule 1 is derived from prior work [55]; the rest are novel strategies specific to this paper for handling the search space introduced by clusters. Following the analysis of prior work, the pruned search space has 11,550 ($\sim 10^4$) possibilities. In contrast, our work, which considers

TABLE IV: Possible partitions for Spatial (S) and Temporal (T) dimensions. The letter combinations in the S and T columns are examples only.

Num of dim in S	**S** (Spatial)	**T** (Temporal)	Num of schedules
1	*M*	*NKL*	$(C_4^1 \times 3! = 24)$
2	*MN*	*KL*	$(C_4^2 \times 2! = 12)$
3	*MNK*	*L*	$(C_4^3 \times 1! = 4)$
4	*MNKL*	$\emptyset$	$(C_4^4 \times 0! = 1)$

Authorized licensed use limited to: INSTITUTE OF COMPUTING TECHNOLOGY CAS. Downloaded on October 13,2026 at 09:14:11 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: Fusion Search Algorithm

Input : Graph g, Device d, Top-k count k
Output: The best execution plan p_{best}

1 **Function** *SearchEngine(g, d, k)*
2 $all_candidates \leftarrow$ EnumerateAllCandidates(g,d);
3 $pruned_candidates \leftarrow$ PruneCandidates$(all_candidates)$;
4 $top_k_list \leftarrow []$;
5 **foreach** (s,t,r) *in pruned_candidates* **do**
6 $(D_v, plan) \leftarrow$ DataflowAnalyzer(g,d,s,t,r);
7 $est_cost \leftarrow$ CalculateCost(D_v);
8 $top_k_list \leftarrow$ UpdateTopKList$(top_k_list, (est_cost, plan), k)$;
9 $p_{best} \leftarrow$ ProfileBestFromList(top_k_list, d);
10 **return** p_{best};

the use of clusters, addresses a much larger search space. Therefore, a cost model is required for further analysis.

3) Search Algorithm: Algorithm 2 details our fusion search method. This algorithm takes a DNN graph `g`, device information `d`, and the top-k count `k` as input. We first employ the pruning strategies mentioned in the previous section to filter the search space (line 3). Then, the legal candidates are fed into the DataflowAnalyzer for detailed analysis. As depicted in Algorithm 1, we analyze and obtain the specific dataflow details under the current parameters, namely the placement of each reused tensor within the cache hierarchy and the concrete data movement volume (line 5-6). Subsequently, using the cost model described in Section IV-C1, we iteratively evaluate each configuration to maintain a list of top-k candidates. Finally, these candidates are profiled on hardware to determine the ultimate execution plan (line 7-9). This entire search is performed offline; at runtime, kernel selection is achieved by using binning and table look-ups for the varying M dimension to select from our pre-compiled kernels. This is efficient because in FFN/conv scenarios, only the M dimension varies dynamically while N, K, and L are fixed.

V. Implementation

FlashFuser is a code generation framework built upon NVIDIA CUTLASS [41]. It takes a high-level DNN model description as input and utilizes our three core components—the Fusion Search Engine, Dataflow Analyzer, and `dsm_comm` primitive—to generate high-performance fused kernels, separating the implementation into a front-end for search and a back-end for code generation.

A. Front-End: The Fusion Search Engine

Our front-end is a Python-based search engine that explores the space of `LoopSchedules`, `TilingSizes` and `ResourceMapping`(with DSM, the lowest-level cache, selected by default). For each configuration, it invokes our Dataflow Analyzer to heuristically determine the memory mapping for intermediate results and compute the data movement volume. It then uses a cost model and pruning rules to filter candidates. The back-end is subsequently invoked to generate code. Finally, the top-K configurations are passed to the hardware for on-device measurement to identify the fused kernel with the optimal performance.

B. Back-End: Code Generation and Primitive Implementation

The back-end translates the optimal plan from the front-end into high-performance CUDA code, leveraging the highly-optimized components of CUTLASS.

a) Realizing the Dataflow Analyzer: Our heuristic plan is realized during code generation. The decision between register and smem is made by calculating the theoretical register usage for a given tile size to avoid performance-degrading spills to global memory. If SMEM is still not large enough, the data must be placed in DSM.

b) Implementing the `dsm_comm` Primitive: We implemented SHUFFLE, MUL, and REDUCE operations for the `dsm_comm` primitive using a fine-grained data exchange mechanism built on TMA for data movement and the `mbarrier` intrinsic for many-to-many synchronization. Unlike the native all-to-one `cluster-sync` in CUTLASS, our `mbarrier`-based approach allows us to synchronize only the necessary groups of CTAs for a given exchange, enabling the construction of higher-level collectives like ring communication for SHUFFLE.

c) Integrating Primitives into Kernel: Our code generator extends the CUTLASS kernel structure—prologue, mainloop, and epilogue—to orchestrate the cluster-level dataflow prescribed by the front-end. In the prologue, semaphore initialization is extended to the DSM to prepare it for inter-CTA communication. The mainloop is augmented with our `dsm_comm` operations. For instance, upon completion of the producer's accumulation loop, a DSM `mul` is performed for GatedFFN variants to exchange and apply computation. Within the consumer's accumulation loop, a DSM `shuffle` implements a ring communication pattern to exchange intermediate results among CTAs. Finally, in the epilogue, a DSM `reduce` accumulates partial sums from different CTAs using a scatter-reduce scheme before storing the final result to global memory. This design maps the problem's spatial dimensions to the grid, while the temporal dimension to the nested execution loop within the kernel's mainloop.

VI. Evaluation

A. Experimental Setup

a) Platforms: Our evaluation is conducted on a server-class accelerator featuring an NVIDIA H100 GPU (SXM). The host system is a dual-socket server equipped with two Intel(R) Xeon(R) Platinum 8468 CPUs (96 cores in total) clocked at 2.10GHz. The primary software stack used in our experiments includes CUDA 12.4, PyTorch 2.6, TVM 0.9, Triton 3.2, and Nsight Compute 2025.2.0.

Authorized licensed use limited to: INSTITUTE OF COMPUTING TECHNOLOGY CAS. Downloaded on October 13,2026 at 09:14:11 UTC from IEEE Xplore. Restrictions apply.

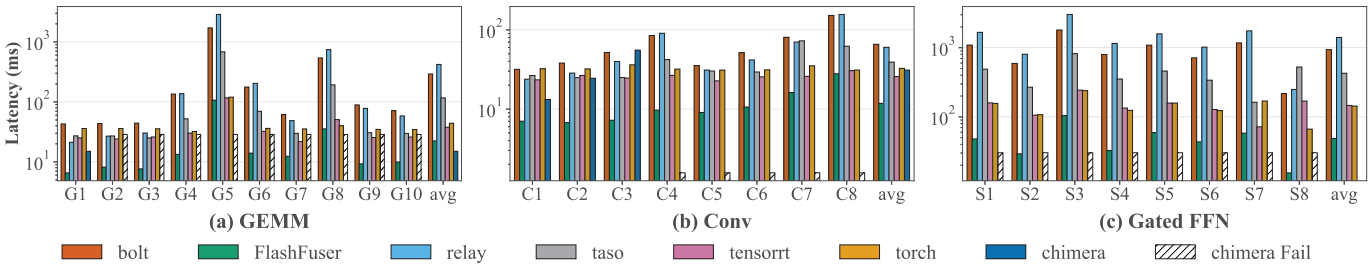


Fig. 10: Performance results in various scenarios: (a) GEMM chains, (b) Convolutional chains, and (c) Gated FFNs.

TABLE V: The configuration of conv chain.

ID	IC	H	W	OC1	OC2	k1	k2
C1	64	56	56	256	64	1	1
C2	128	28	28	512	128	1	1
C3	256	14	14	1024	256	1	1
C4	512	7	7	2048	512	1	1
C5	64	56	56	64	256	3	1
C6	128	28	28	128	512	3	1
C7	256	14	14	256	1024	3	1
C8	512	7	7	512	2048	3	1

TABLE VI: The configuration of gated FFN.

ID	m	n	k	l	Model
S1	128	8192	3072	3072	llama-3.2-3B
S2	128	5632	2048	2048	llama-1.1B
S3	128	11008	4096	4096	Llama-2-7b
S4	128	8192	2048	2048	Qwen2.5-2.1B
S5	128	11008	2048	2048	Qwen2.5-3B
S6	128	8960	1536	1536	Qwen2.5-1.5B
S7	128	9728	2560	2560	Qwen3-4B
S8	128	3072	1024	1024	Qwen3-0.6B

b) Baselines: We compare FlashFuser against a comprehensive set of baselines, covering industry-standard libraries and state-of-the-art research compilers.

Libraries: We compare against PyTorch [35] 2.6 (which utilizes cuBLAS for its GEMM implementation) and NVIDIA's TensorRT [31], a highly optimized inference engine. For the PyTorch baseline, we enable `torch.compile`, which significantly reduces kernel launch overhead.

Compilers: We select several state-of-the-art machine learning compilers, including relay [39], TASO [17], BOLT [51], and Chimera [60]. TVM/Relay [39] effectively fuses kernels with a compute-activation pattern. TASO automatically performs subgraph substitutions, replacing parts of the graph with functionally equivalent but more performant alternatives (e.g., reordering consecutive matrix multiplications), but it does not support the fusion of compute-intensive operators. BOLT fuses consecutive GEMMs based on using smem and reg. Chimera implements fusion for consecutive GEMMs while also exploring different block execution orders.

c) Subgraph Configurations: The configurations of the subgraphs are detailed in Tables VII, VI, and V. In Tables VII [13], [21], [43] and VI, the dimensions of GEMM1 are $(m \times n \times k)$ and GEMM2 are $(m \times l \times n)$. In Table V, the dimensions are $(IC, H, W) \times (OC1, IC, K1, K1)$ for conv1 and $(OC1, H, W) \times (OC2, OC1, K2, K2)$ for conv2, where `OC1` and `OC2` are the output channel sizes of conv1 and conv2, respectively; `H` and `W` are the height and width of the feature map; and `K1` and `K2` are the respective kernel sizes.

TABLE VII: The configuration of gemm chain.

ID	m	n	k	l	Model
G1	128	512	32	256	DLRM-0
G2	128	256	512	64	DLRM-1
G3	128	512	416	256	DLRM-2
G4	128	3072	768	768	GPT-2-Small
G5	128	16384	4096	4096	GPT-6.7B
G6	128	4096	1024	1024	GPT2-medium
G7	128	768	768	768	nlp_gpt3_base
G8	128	8192	2048	2048	OPT-1.3B
G9	128	2048	512	512	Performer
G10	128	1536	384	384	BERT

B. Subgraph Performance

a) Performance Results: The performance evaluation results for GEMM and convolution chains are presented in Figure 10, with performance normalized to PyTorch.

b) GEMM Chains: In the GEMM chain scenario, FlashFuser achieves significant speedups over all baselines, with average speedups of 5.4x over BOLT, 4.6x over Chimera, 4.7x over Relay, 3.4x over TASO, 2.4x over TensorRT, and 3.1x over PyTorch. Although compilers like BOLT and Chimera also perform operator fusion, their methods have inherent limitations. Chimera's fusion capability is strictly limited by the SMEM size, causing it to fail on configurations with large intermediate tensors. BOLT utilizes CUTLASS templates within TVM but is constrained by its fixed block execution order, which may not be optimal. In contrast, FlashFuser's analytical model can explore a more diverse range of block execution orders. Other baselines like TASO and Relay do not fuse the two GEMMs, leading to separate kernel launches and additional global memory access overhead. Crucially, none of the above baselines leverage DSM, which fundamentally restricts their fusion scope. FlashFuser overcomes these limitations by using DSM to expand the fusion boundary.

Authorized licensed use limited to: INSTITUTE OF COMPUTING TECHNOLOGY CAS. Downloaded on October 13,2026 at 09:14:11 UTC from IEEE Xplore. Restrictions apply.

c) Convolution Chains: For convolution chains extracted from real-world ResNet models, FlashFuser achieves average speedups of 6.3x over BOLT, 6.4x over Chimera, 5.6x over Relay, 4.3x over TASO, 3.3x over TensorRT, and 3.9x over PyTorch. For smaller problem sizes, BOLT performs kernel fusion to achieve significant performance gains. However, when the problem sizes become large, BOLT abandons fusion, resulting in comparatively poorer performance. Chimera fails when convolution sizes become too large. Other baselines execute independent, non-fused convolution kernels. FlashFuser utilizes DSM as a larger on-chip buffer to expand the scope of fusible operations, resulting in substantial performance gains.

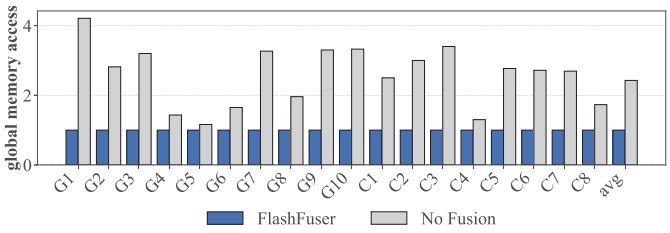


Fig. 11: Comparison of global memory access between FlashFuser and PyTorch.

C. Performance Analysis

To verify the source of the observed performance gains, we profiled the generated kernels using NVIDIA's Nsight Compute, focusing on memory access patterns. As shown in Figure 11, FlashFuser significantly reduces global memory access compared to non-fused approaches like PyTorch. The analysis indicates that PyTorch, due to its lack of fusion, writes intermediate results to global memory before reading them back into shared memory for the next operator. In contrast, FlashFuser enables data reuse at higher levels of the memory hierarchy, including DSM. On average, PyTorch kernels exhibit 2.4× more global memory traffic than FlashFuser kernels, confirming that reduced off-chip memory access is a primary source of our acceleration.

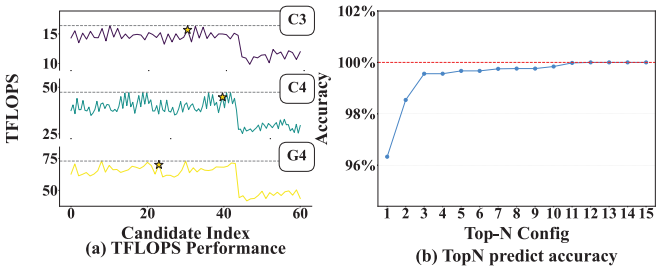


Fig. 12: Validation of cost model and Analysis of top-K.

To validate our cost model and search strategy, we evaluate its capability to identify optimal configurations, the selection of an appropriate `topK` value, and the compilation time overhead. Figure 12a illustrates the search efficacy across the C3, C4, and G4 benchmarks. In the figure, the vertical axis represents the computing performance in TFLOPS, and different colored lines denote different models. The star markers indicate the configurations selected by our cost model. The results demonstrate that our cost model consistently identifies the performance-optimal or near-optimal configurations. Our analysis of `topK` selection (Figure. 12b), using data from Table VII and Table V, computes accuracy as the average ratio of predicted performance to the true optimal performance. The figure shows that performance approaches 100% as K increases beyond 11, making `K=11` our chosen value. Furthermore, our search engine accelerates compilation by 12–864× compared to a brute-force search (Table VIII), demonstrating its efficiency. This overhead primarily consists of the cost model's prediction (typically 1-2s) and the compilation time for the top-K kernels. This highlights the importance of selecting an appropriate K.

TABLE VIII: Search Time Comparison (search engine (TopK=11) vs. Brute-Force).

	Brute-Force Time	Search-Engine Time	Speedup
G3	1.2 hr	362.1 s	12.25×
G4	3.0 hr	380.3 s	29.05×
G5	8.1 hr	381.0 s	68.26×

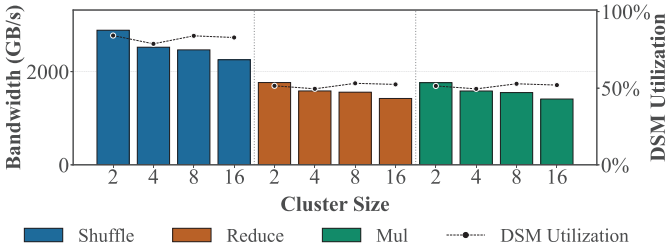


Fig. 13: Bandwidth and its utilization of `dsm_comm` primitive

To validate the performance of our three proposed `dsm_comm` primitives, we measured their bandwidth and utilization across different cluster sizes. The benchmark transfers a 32768×32768 tensor, slicing it into 128x128 tiles to execute `dsm_comm` operations within the cluster (excluding global read/store overhead), which is looped 1000 times to measure the bandwidth. Bandwidth utilization is calculated by dividing the measured bandwidth by the peak DSM bandwidth for the corresponding cluster size. As shown in Figure. 13, while the bandwidth decreases as the cluster size increases, the bandwidth utilization remains stable. The `Shuffle` primitive outperforms `Reduce` and `Mul` because the latter two incur computational overhead in addition to data transfer.

We conduct a detailed ablation study on our three key designs: `dsm_comm` (DC), dataflow analyzer (DA), and search engine (SE). We evaluate the full system ('All'), 'DC+DA' (using a random configuration), and 'DA' (using only SMEM/-global memory for fusion). As shown in Figure. 15, compared to a no-fusion baseline, the 'All', 'DC+DA', and 'DA' config-

Authorized licensed use limited to: INSTITUTE OF COMPUTING TECHNOLOGY CAS. Downloaded on October 13,2026 at 09:14:11 UTC from IEEE Xplore. Restrictions apply.

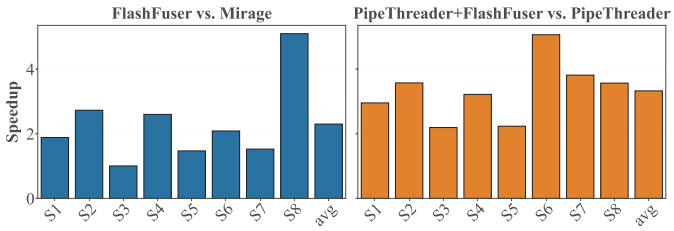


Fig. 14: Comparison to mirage and pipethreader.

urations yield speedups of 3.29×, 2.11×, and 1.52×, respectively. This demonstrates the effectiveness of our methods.

We evaluate our end-to-end inference performance against the SGLang framework on a suite of real-world models (Table VII/VI). As illustrated in Figure. 17, our approach achieves an average performance improvement of 1.32×. We further extend our evaluation to larger models and input sizes in Figure. 16, testing Llama3-70B, Qwen2.5-14B and 32B. Figure 16a presents a roofline analysis, which indicates that these models are primarily compute-bound, thus offering limited room for kernel-level optimization. In Figure 16b, we showcase the E2E speedup. For this setup, we fix the sequence length at 256 and change batch size from 1 to 32. Across these configurations, our kernel achieves an average performance improvement of 1.22×, leading to an average E2E speedup of 1.16×. When considering all scenarios, including both small and large inputs, the overall E2E speedup reaches 1.24×.

While our evaluation is conducted on the NVIDIA H100, the proposed fusion strategy is not limited to a specific architecture. FlashFuser's core abstraction, `dsm_comm`, is a topology-agnostic collective communication concept at the design level. At the implementation level, for architectures with crossbar interconnects (e.g., Graphcore IPU [20], H100), our approach is directly applicable. For mesh architectures (e.g., Cerebras WSE [29]), a potential mapping distributes shuffle groups (defined in §IV-A) to neighboring cores to perform shuffle and reduce operations.

VII. Related Work

While extensive research exists in both kernel fusion and Distributed Shared Memory (DSM), the intersection of these fields–how to perform efficient, automated kernel fusion on modern GPUs with DSM–remains largely explored.

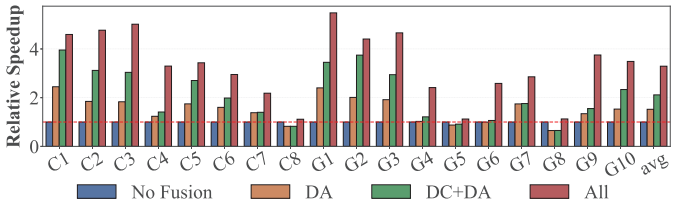


Fig. 15: Ablation study of FlashFuser by Isolating the Contributions of Search Engine (SE), `dsm_comm` (DC), and Dataflow Analyzer (DA)

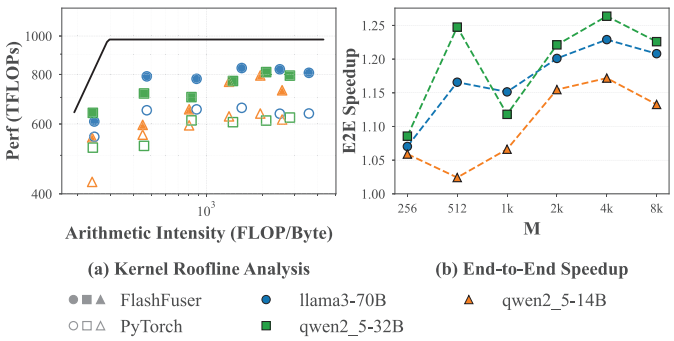


Fig. 16: Kernel performance and end-to-end performance of larger LLM.

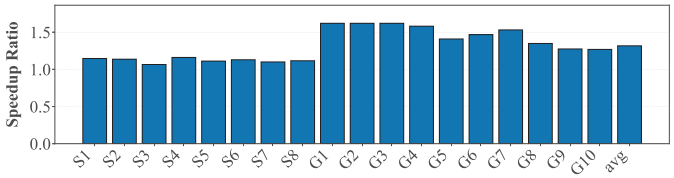


Fig. 17: End-to-end performance evaluation based on SGLang.

A. Research on Kernel Fusion

The development of kernel fusion [50], [59], [68], a key compiler optimization, can be broadly categorized by the types of operators being fused.

The first primary category of fusion pairs a compute-intensive operator with subsequent memory-intensive consumers (e.g., activations, bias additions). *Halide* [37] pioneered this for image processing pipelines with powerful schedule primitives, although for operators less complex than typical GEMMs or convolutions. Modern compilers like *TVM* [3] and *Ansor* [56] advanced this by transforming loop nests to keep intermediate data in registers. To further expand the fusion scope, works like *Fusion Stitching* [63] and *AStitch* [62] used shared memory as an intermediate buffer to fuse operators.

Another category is the fusion of compute-intensive operator chains (e.g., GEMM → GEMM). *BOLT* [51] matches common patterns and invokes optimized *Cutlass* [41] templates, though it is limited by the fixed loop schedules of Cutlass. More general transformation-based approaches include *TASO* [17], which employs graph substitution to combine convolutions that can run in parallel, yet it lacks the capability to fuse sequential convolutions, and *Chimera* [60], which optimizes at a finer grain by rescheduling dataflow between thread blocks to maximize locality.

However, a common limitation across all these works is their confinement to the resources of a single SM. This reliance forces fusion to fail when intermediate results exceed `smem`'s limited capacity. To overcome this problem, emerging hardware features like DSM have been introduced to expand the on-chip memory space.

Authorized licensed use limited to: INSTITUTE OF COMPUTING TECHNOLOGY CAS. Downloaded on October 13,2026 at 09:14:11 UTC from IEEE Xplore. Restrictions apply.

B. Research on DSM

The study of DSM has gained traction in recent years. Researchers have explored how to design and utilize its features through various approaches, including architectural simulations and performance studies on specialized hardware.

Some research focuses on architectural exploration through simulation, proposing novel mechanisms for inter-core data sharing. For instance, Ibrahim et al. [15] proposed a "shared L1" organization to reduce redundant data replication on different L1 caches and analyzed which applications benefit from this data sharing. Falahati et al. [6] also interconnected L1 caches and used a predictor to determine if a cache block exists in another SM.

Other studies involve performance explorations on physical hardware that incorporates DSM. The Graphcore IPU, targeted by *T10* [22], has a GPU-like crossbar `smem` interconnection but assumes no HBM, a key difference from modern GPUs. The Cerebras processor, targeted by *WaferLLM* [12], uses a mesh interconnect L1 cache, which differs from standard GPU topology. Thus, these works have two limitations: their conclusions are not directly transferable to mainstream GPUs, and they typically focus on single-operator scheduling scenarios. Additionally, *ClusterFusion* [27] explores utilizing DSM for kernel fusion on GPUs; however, it focuses on hand-written kernels and lacks a compiler-based method for parameter selection and code generation.

While these studies highlight the potential of inter-core data sharing, a systematic compilation framework for modern GPUs is still missing. Interestingly, the concept of leveraging inter-core connections for dataflow—relatively new to general-purpose GPUs—has long been a foundational design principle in domain-specific spatial architectures.

C. Fusion on Spatial Architectures

Research on kernel fusion for specialized spatial architectures (e.g., ASIC accelerators and systolic arrays) primarily focuses on leveraging explicit on-chip Networks-on-Chip (NoC) between Processing Elements (PEs) to construct efficient dataflows. *FLAT* [19] targets memory bottlenecks in Transformer models by proposing a "Fixed-Loop-Aligning Tiling" strategy. It utilizes direct data reuse between PEs in a spatial array to stage intermediate results in on-chip buffers, thereby fusing originally discrete operators into a pipelined execution. *COMET* [30] introduces primitives containing explicit collectives to formally model the dataflow of compound operations, supporting the mapping of complex fusion patterns. Additionally, *DESA* [46] designs a dataflow-efficient systolic array that achieves fully fused attention computation by decoupling computation from data transfer. While these works demonstrate the efficacy of spatial dataflow, they typically rely on specific hardware interconnect topologies or systolic array structures. In contrast, FlashFuser targets on GPU. It exploits the emerging DSM mechanism on modern GPUs (e.g., NVIDIA H100) to enable direct inter-core communication.

D. Emerging GPU Compilers and DSLs

To facilitate efficient code generation and optimize dataflow on GPUs, extensive research has been dedicated to machine learning compilation and Domain-Specific Languages (DSLs) [7], [28], [36], [48], [49], [53], [54], [58], [61], [62], [67].

Notably, *Triton* [42] and its derivatives simplify high-performance kernel development through a block-based programming model and have been widely adopted for operator fusion. The recently proposed *TileLang* [44] (and its underlying low-precision library *Ladder* [45]) advances this direction by proposing a composable tiled language and hardware-aware tensor transformations. These tools allow developers to explicitly define parallel tiling strategies and pipeline schedules across multiple memory levels via a Python interface. Although these DSLs offer powerful representation capabilities, they primarily focus on the traditional memory hierarchy and often rely on expert users to manually specify scheduling strategies. FlashFuser distinguishes itself by integrating DSM into the compiler's automated search space.

VIII. Conclusion

In this paper, we presented FlashFuser, the first compiler framework that overcomes this limitation by leveraging the inter-core connection capabilities of modern GPUs. By introducing a DSM communication abstraction, using a dataflow analyzer to evaluate data placement and costs, and leveraging an efficient search engine to explore the vast search space, FlashFuser systematically generates highly efficient fused kernels. On an NVIDIA H100 GPU, our evaluation shows that FlashFuser delivers kernel speedups of up to 3.3× against highly-tuned libraries and 4.1× against state-of-the-art compilers. These gains, driven by a 58% reduction in memory access, lead to a 1.24× end-to-end speedup.

Acknowledgment

We thank Dr. Size Zheng for providing the source code of Chimera. This work was supported by the National Key R&D Program of China under Grant 2022YFB4501400, and the National Natural Science Foundation of China (NSFC) Grants (62222210 and 62532006) and Shanghai Qi Zhi Institute Innovation Program SQZ202316. Any opinions, findings, and conclusions in this paper are those of the authors only and do not necessarily reflect the views of our sponsors.

References

[1] H. Abdelkhalik, Y. Arafa, N. Santhi, and A.-H. A. Badawy, "Demystifying the nvidia ampere architecture through microbenchmarking and instruction-level analysis," in *2022 IEEE High Performance Extreme Computing Conference (HPEC)*. Ieee, 2022, pp. 1–8.

[2] R. Chen, Z. Ding, S. Zheng, C. Zhang, J. Leng, X. Liu, and Y. Liang, "Magis: Memory optimization via coordinated graph transformation and scheduling for dnn," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 607–621.

[3] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze *et al.*, "{TVM}: An automated {End-to-End} optimizing compiler for deep learning," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018, pp. 578–594.

Authorized licensed use limited to: INSTITUTE OF COMPUTING TECHNOLOGY CAS. Downloaded on October 13,2026 at 09:14:11 UTC from IEEE Xplore. Restrictions apply.

[4] J. Choquette, "Nvidia hopper gpu: Scaling performance," in *2022 IEEE Hot Chips 34 Symposium (HCS)*. IEEE r Society, 2022, pp. 1–46.

[5] J. Choquette and W. Gandhi, "Nvidia a100 gpu: Performance & innovation for gpu computing," in *2020 IEEE Hot Chips 32 Symposium (HCS)*. IEEE r Society, 2020, pp. 1–43.

[6] H. Falahati, M. Sadrosadati, Q. Xu, J. Gómez-Luna, B. Saber Latibari, H. Jeon, S. Hesaabi, H. Sarbazi-Azad, O. Mutlu, M. Annavaram *et al.*, "Cross-core data sharing for energy-efficient gpus," *ACM Transactions on Architecture and Code Optimization*, vol. 21, no. 3, pp. 1–32, 2024.

[7] S. Feng, B. Hou, H. Jin, W. Lin, J. Shao, R. Lai, Z. Ye, L. Zheng, C. H. Yu, Y. Yu *et al.*, "Tensorir: An abstraction for automatic tensorized program optimization," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2023, pp. 804–817.

[8] Y. Guan, Y. Qiu, J. Leng, F. Yang, S. Yu, Y. Liu, Y. Feng, Y. Zhu, L. Zhou, Y. Liang *et al.*, "Amanda: Unified instrumentation framework for deep neural networks," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2024, pp. 1–18.

[9] Y. Guan, C. Yu, Y. Zhou, J. Leng, C. Li, and M. Guo, "Fractal: Joint multi-level sparse pattern tuning of accuracy and performance for dnn pruning," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 416–430.

[10] C. Guo, J. Tang, W. Hu, J. Leng, C. Zhang, F. Yang, Y. Liu, M. Guo, and Y. Zhu, "Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–15.

[11] C. Guo, R. Zhang, J. Xu, J. Leng, Z. Liu, Z. Huang, M. Guo, H. Wu, S. Zhao, J. Zhao *et al.*, "Gmlake: Efficient and transparent gpu memory defragmentation for large-scale dnn training with virtual memory stitching," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2024, pp. 450–466.

[12] C. He, Y. Huang, P. Mu, Z. Miao, J. Xue, L. Ma, F. Yang, and L. Mai, "Waferllm: Large language model inference at wafer scale," *arXiv preprint arXiv:2502.04563*, 2025.

[13] M. Hildebrand, J. Lowe-Power, and V. Akella, "Efficient large scale dlrm implementation on heterogeneous memory systems," in *International Conference on High Performance Computing*. Springer, 2023, pp. 42–61.

[14] W. Hu, H. Zhang, C. Guo, Y. Feng, R. Guan, Z. Hua, Z. Liu, Y. Guan, M. Guo, and J. Leng, "M-ant: Efficient low-bit group quantization for llms via mathematically adaptive numerical type," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1112–1126.

[15] M. A. Ibrahim, O. Kayiran, Y. Eckert, G. H. Loh, and A. Jog, "Analyzing and leveraging shared l1 caches in gpus," in *Proceedings of the ACM International Conference on Parallel Architectures and Compilation Techniques*, 2020, pp. 161–173.

[16] ——, "Analyzing and leveraging decoupled l1 caches in gpus," in *2021 IEEE International Symposium on High-Performance r Architecture (HPCA)*. IEEE, 2021, pp. 467–478.

[17] Z. Jia, O. Padon, J. Thomas, T. Warszawski, M. Zaharia, and A. Aiken, "Taso: optimizing deep learning computation with automatic generation of graph substitutions," in *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, 2019, pp. 47–62.

[18] Z. Jin, C. Rocca, J. Kim, H. Kasan, M. Rhu, A. Bakhoda, T. M. Aamodt, and J. Kim, "Uncovering real gpu noc characteristics: Implications on interconnect architecture," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 885–898.

[19] S.-C. Kao, S. Subramanian, G. Agrawal, A. Yazdanbakhsh, and T. Krishna, "Flat: An optimized dataflow for mitigating attention bottlenecks," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2023, pp. 295–310.

[20] S. Knowles, "Graphcore," in *2021 IEEE Hot Chips 33 Symposium (HCS)*. IEEE, 2021, pp. 1–25.

[21] M. V. Koroteev, "Bert: a review of applications in natural language processing and understanding," *arXiv preprint arXiv:2103.11943*, 2021.

[22] Y. Liu, Y. Xue, Y. Cheng, L. Ma, Z. Miao, J. Xue, and J. Huang, "Scaling deep learning computation over the inter-core connected intelligence processor with t10," in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, 2024, pp. 505–521.

[23] Z. Liu, X. Luo, J. Guo, W. Ni, Y. Zhou, Y. Guan, C. Guo, W. Cui, Y. Feng, M. Guo *et al.*, "Vq-llm: High-performance code generation for vector quantization augmented llm inference," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1496–1509.

[24] Z. Liu, W. Ni, J. Leng, Y. Feng, C. Guo, Q. Chen, C. Li, M. Guo, and Y. Zhu, "Juno: optimizing high-dimensional approximate nearest neighbour search with sparsity-aware algorithm and ray-tracing core mapping," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2024, pp. 549–565.

[25] T. Lühnen, T. Marschner, and S. Lal, "Benchmarking thread block cluster," in *2024 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 2024, pp. 1–7.

[26] W. Luo, R. Fan, Z. Li, D. Du, Q. Wang, and X. Chu, "Benchmarking and dissecting the nvidia hopper gpu architecture," in *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2024, pp. 656–667.

[27] X. Luo, Z. Liu, Y. Zhou, S. Fang, Z. Huang, Y. Feng, C. Zhang, S. Sun, Z. Zheng, J. Leng *et al.*, "Clusterfusion: Expanding operator fusion scope for llm inference via cluster-level collective primitive," *arXiv e-prints*, pp. arXiv–2508, 2025.

[28] L. Ma, Z. Xie, Z. Yang, J. Xue, Y. Miao, W. Cui, W. Hu, F. Yang, L. Zhang, and L. Zhou, "Rammer: Enabling holistic deep learning compiler optimizations with {rTasks}," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020, pp. 881–897.

[29] R. Matsuzaki, D. Mukunoki, and T. Miyajima, "Performance evaluation and modelling of single-precision matrix multiplication on cerebras cs-2," in *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2024, pp. 727–731.

[30] S. Negi, M. Singhal, A. Ankit, S. Bhoja, and K. Roy, "Comet: A framework for modeling compound operation dataflows with explicit collectives," *arXiv preprint arXiv:2509.00599*, 2025.

[31] NVIDIA, "TensorRT: A High-Performance Deep Learning Inference SDK," https://github.com/NVIDIA/TensorRT, 2025, version 10.13 OSS. Accessed: July 28, 2025.

[32] NVIDIA Co. (2025) Nvidia cublas. NVIDIA Corporation. [Online]. Available: https://developer.nvidia.com/cublas

[33] NVIDIA Corp. (2025) Cuda c++ programming guide - 4.6.10. distributed shared memory. NVIDIA Corporation. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/#distributed-shared-memory

[34] NVIDIA Corporation, *CUDA C++ Programming Guide*, 2025, accessed: 2025-11-25. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/

[35] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, 2019.

[36] P. M. Phothilimthana, A. S. Elliott, A. Wang, A. Jangda, B. Hagedorn, H. Barthels, S. J. Kaufman, V. Grover, E. Torlak, and R. Bodik, "Swizzle inventor: data movement synthesis for gpu kernels," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2019, pp. 65–78.

[37] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, "Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines," *Acm Sigplan Notices*, vol. 48, no. 6, pp. 519–530, 2013.

[38] S. Riedel, M. Cavalcante, R. Andri, and L. Benini, "Mempool: A scalable manycore architecture with a low-latency shared l1 memory," *IEEE Transactions on Computers*, vol. 72, no. 12, pp. 3561–3575, 2023.

[39] J. Roesch, S. Lyubomirsky, L. Weber, J. Pollock, M. Kirisame, T. Chen, and Z. Tatlock, "Relay: A new ir for machine learning frameworks," in *Proceedings of the 2nd ACM SIGPLAN international workshop on machine learning and programming languages*, 2018, pp. 58–68.

[40] Y. Shi, Z. Yang, J. Xue, L. Ma, Y. Xia, Z. Miao, Y. Guo, F. Yang, and L. Zhou, "Welder: Scheduling deep learning memory access via tile-graph," in *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, 2023, pp. 701–718.

Authorized licensed use limited to: INSTITUTE OF COMPUTING TECHNOLOGY CAS. Downloaded on October 13,2026 at 09:14:11 UTC from IEEE Xplore. Restrictions apply.

[41] V. Thakkar, P. Ramani, C. Cecka, A. Shivam, H. Lu, E. Yan, J. Kosaian, M. Hoemmen, H. Wu, A. Kerr, M. Nicely, D. Merrill, D. Blasig, F. Qiao, P. Majcher, P. Springer, M. Hohnerbach, J. Wang, and M. Gupta, "CUTLASS," Jan. 2023. [Online]. Available: https://github.com/NVIDIA/cutlass

[42] P. Tillet, H.-T. Kung, and D. Cox, "Triton: an intermediate language and compiler for tiled neural network computations," in *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019, pp. 10–19.

[43] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[44] L. Wang, Y. Cheng, Y. Shi, Z. Tang, Z. Mo, W. Xie, L. Ma, Y. Xia, J. Xue, F. Yang *et al.*, "Tilelang: A composable tiled programming model for ai systems," *arXiv preprint arXiv:2504.17577*, 2025.

[45] L. Wang, L. Ma, S. Cao, Q. Zhang, J. Xue, Y. Shi, N. Zheng, Z. Miao, F. Yang, T. Cao *et al.*, "Ladder: Enabling efficient {Low-Precision} deep learning computing through hardware-aware tensor transformation," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 307–323.

[46] Z. Wang, H. Fan, and G. He, "Desa: Dataflow efficient systolic array for acceleration of transformers," *IEEE Transactions on Computers*, 2025.

[47] X. Wei, S. Moalla, R. Pascanu, and C. Gulcehre, "Building on efficient foundations: Effective training of llms with structured feedforward layers," *Advances in Neural Information Processing Systems*, vol. 37, pp. 4689–4717, 2024.

[48] J. Weng, A. Jain, J. Wang, L. Wang, Y. Wang, and T. Nowatzki, "Unit: Unifying tensorized instruction compilation," in *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2021, pp. 77–89.

[49] X. Wu, P. Paramasivam, and V. Taylor, "Autotuning apache tvm-based scientific applications using bayesian optimization," in *Proceedings of the SC'23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*, 2023, pp. 29–35.

[50] C. Xia, J. Zhao, Q. Sun, Z. Wang, Y. Wen, T. Yu, X. Feng, and H. Cui, "Optimizing deep learning inference via global analysis and tensor expressions," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2024, pp. 286–301.

[51] J. Xing, L. Wang, S. Zhang, J. Chen, A. Chen, and Y. Zhu, "Bolt: Bridging the gap between auto-tuners and hardware-native performance," *Proceedings of Machine Learning and Systems*, vol. 4, pp. 204–216, 2022.

[52] J. Xu, R. Zhang, C. Guo, W. Hu, Z. Liu, F. Wu, Y. Feng, S. Sun, C. Shao, Y. Guo *et al.*, "vtensor: Flexible virtual tensor management for efficient llm serving," *arXiv preprint arXiv:2407.15309*, 2024.

[53] Y. Zhai, Y. Zhang, S. Liu, X. Chu, J. Peng, J. Ji, and Y. Zhang, "Tlp: A deep learning-based cost model for tensor program tuning," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2023, pp. 833–845.

[54] C. Zhang, L. Ma, J. Xue, Y. Shi, Z. Miao, F. Yang, J. Zhai, Z. Yang, and M. Yang, "Cocktailer: Analyzing and optimizing dynamic control flow in deep learning," in *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, 2023, pp. 681–699.

[55] Z. Zhang, D. Yang, X. Zhou, and D. Cheng, "Mcfuser: High-performance and rapid fusion of memory-bound -intensive operators," in *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2024, pp. 1–15.

[56] L. Zheng, C. Jia, M. Sun, Z. Wu, C. H. Yu, A. Haj-Ali, Y. Wang, J. Yang, D. Zhuo, K. Sen *et al.*, "Ansor: Generating {High-Performance} tensor programs for deep learning," in *14th USENIX symposium on operating systems design and implementation (OSDI 20)*, 2020, pp. 863–879.

[57] L. Zheng, L. Yin, Z. Xie, C. L. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez *et al.*, "Sglang: Efficient execution of structured language model programs," *Advances in neural information processing systems*, vol. 37, pp. 62 557–62 583, 2024.

[58] S. Zheng, R. Chen, A. Wei, Y. Jin, Q. Han, L. Lu, B. Wu, X. Li, S. Yan, and Y. Liang, "Amos: enabling automatic mapping for tensor computations on spatial accelerators with hardware abstraction," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 874–887.

[59] S. Zheng, S. Chen, S. Gao, L. Jia, G. Sun, R. Wang, and Y. Liang, "Tileflow: A framework for modeling fusion dataflow via tree-based analysis," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 1271–1288.

[60] S. Zheng, S. Chen, P. Song, R. Chen, X. Li, S. Yan, D. Lin, J. Leng, and Y. Liang, "Chimera: An analytical optimizing framework for effective -intensive operators fusion," in *2023 IEEE International Symposium on High-Performance r Architecture (HPCA)*. IEEE, 2023, pp. 1113–1126.

[61] S. Zheng, Y. Liang, S. Wang, R. Chen, and K. Sheng, "Flextensor: An automatic schedule exploration and optimization framework for tensor computation on heterogeneous system," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020, pp. 859–873.

[62] Z. Zheng, X. Yang, P. Zhao, G. Long, K. Zhu, F. Zhu, W. Zhao, X. Liu, J. Yang, J. Zhai *et al.*, "Astitch: enabling a new multi-dimensional optimization space for memory-intensive ml training and inference on modern simt architectures," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 359–373.

[63] Z. Zheng, P. Zhao, G. Long, F. Zhu, K. Zhu, W. Zhao, L. Diao, J. Yang, and W. Lin, "Fusionstitching: boosting memory intensive computations for deep learning workloads," *arXiv preprint arXiv:2009.10924*, 2020.

[64] Y. Zhou, J. Leng, Y. Song, S. Lu, M. Wang, C. Li, M. Guo, W. Shen, Y. Li, W. Lin *et al.*, "ugrapher: High-performance graph operator computation via unified abstraction for graph neural networks," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2023, pp. 878–891.

[65] Y. Zhou, W. Shen, J. Leng, S. Lu, Z. Liu, W. Cui, Z. Zhang, W. Xiao, B. Ai, Y. Li *et al.*, "Voyager: Input-adaptive algebraic transformations for high-performance graph neural networks," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2025, pp. 247–263.

[66] Y. Zhou, H. Zhu, Q. Qiu, W. Cui, Z. Liu, P. Chen, M. Wahib, C. Guo, S. Feng, J. Meng *et al.*, "A sample-free compilation framework for efficient dynamic tensor computation," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2025, pp. 167–184.

[67] H. Zhu, R. Wu, Y. Diao, S. Ke, H. Li, C. Zhang, J. Xue, L. Ma, Y. Xia, W. Cui *et al.*, "{ROLLER}: Fast and efficient tensor compilation for deep learning," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 233–248.

[68] D. Zhuang, Z. Zheng, H. Xia, X. Qiu, J. Bai, W. Lin, and S. L. Song, "{MonoNN}: Enabling a new monolithic optimization space for neural network inference tasks on modern {GPU-Centric} architectures," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 989–1005.

Authorized licensed use limited to: INSTITUTE OF COMPUTING TECHNOLOGY CAS. Downloaded on October 13,2026 at 09:14:11 UTC from IEEE Xplore. Restrictions apply.